PocketSmart AI

Main Functionality Code Routings

FastAPI Backend — Income, Expense & Net Savings Endpoints

Name	Divya Dharshini S
Project Title	PocketSmart AI
Component	Backend Routing Logic
Technology Stack	Python 3, FastAPI, Pydantic, Google Gemini API

1. Overview of Routing Logic

The core business functionality of PocketSmart AI is exposed through a single FastAPI APIRouter, mounted under the /api prefix. This module is responsible for three related concerns: strictly validating incoming financial data using Pydantic models, performing deterministic arithmetic to compute income, expenditure, and net savings, and orchestrating the call to the Gemini service to obtain AI-generated financial guidance. Keeping these concerns in one cohesive router file, while delegating the actual Gemini call to a separate service module, follows the single-responsibility principle and keeps the codebase easy to test and extend.

Two Pydantic models — ExpenseItem and BudgetRequest — describe the exact shape of data the frontend must send, while a third model, BudgetResponse, guarantees that every successful response follows a consistent, machine-readable structure. FastAPI uses these models to auto-generate input validation, error messages, and OpenAPI/Swagger documentation with no additional code.

1.1 Endpoint Summary

- POST /api/analyze — accepts income and expenses, returns computed totals plus AI advice.
- GET /api/health — lightweight liveness check used by the frontend on page load.

2. Production-Ready Code

The following module (routers/budget.py) implements the routing and calculation logic described above:

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from typing import List

router = APIRouter(prefix="/api", tags=["Budget"])


# ---------------------------------------------------------------------
# Pydantic Models — define and validate the shape of incoming/outgoing data
# ---------------------------------------------------------------------

class ExpenseItem(BaseModel):
    """A single categorised expense entry."""
    category: str = Field(..., example="Groceries")
    amount: float = Field(..., gt=0, example=2500.0)
```

```
class BudgetRequest(BaseModel):
    """Payload received from the frontend when a user submits their budget."""
    monthly_income: float = Field(..., gt=0, example=45000.0)
    expenses: List[ExpenseItem]
    query: str = Field(
        default="Give me tips to save more this month.",
        description="Optional free-text question for the AI assistant."
    )


class BudgetResponse(BaseModel):
    """Structured response returned to the frontend."""
    total_income: float
    total_expenses: float
    net_savings: float
    savings_rate_percent: float
    ai_advice: str


# ---------------------------------------------------------------------
# Core calculation logic — pure functions, independent of the web layer
# ---------------------------------------------------------------------

def calculate_totals(payload: BudgetRequest) -> dict:
    """Compute total expenses and net savings from a validated request."""
    total_expenses = sum(item.amount for item in payload.expenses)
    net_savings = payload.monthly_income - total_expenses

    # Guard against division by zero when income is not yet set
    savings_rate = (
        (net_savings / payload.monthly_income) * 100
        if payload.monthly_income > 0 else 0.0
    )

    return {
        "total_income": payload.monthly_income,
        "total_expenses": round(total_expenses, 2),
        "net_savings": round(net_savings, 2),
        "savings_rate_percent": round(savings_rate, 2),
    }


# ---------------------------------------------------------------------
# Route: POST /api/analyze
# Accepts income + expenses, computes totals, and returns AI-backed advice
# ---------------------------------------------------------------------

@router.post("/analyze", response_model=BudgetResponse)
async def analyze_budget(payload: BudgetRequest):
    """
    Main routing endpoint for PocketSmart AI.

    1. Validates the incoming JSON against BudgetRequest.
    2. Calculates income, expenses and net savings.
    3. Sends a contextual prompt to Gemini for personalised advice.
    4. Returns a single consolidated JSON response.
    """
    try:
        totals = calculate_totals(payload)

        # Import kept local to this call to avoid circular imports
        # with the Gemini helper module (see gemini_service.py)
        from services.gemini_service import get_financial_advice

        ai_advice = await get_financial_advice(
            income=totals["total_income"],
```

```
            expenses=totals["total_expenses"],
            savings=totals["net_savings"],
            user_query=payload.query,
        )

        return BudgetResponse(**totals, ai_advice=ai_advice)

    except Exception as exc:
        # Convert any unexpected failure (e.g. Gemini timeout) into a
        # clean, client-friendly HTTP error instead of a raw 500 trace
        raise HTTPException(
            status_code=502,
            detail=f"Unable to generate budget analysis: {exc}"
        )


# ---------------------------------------------------------------------
# Route: GET /api/health
# Lightweight route used to confirm the API is reachable
# ---------------------------------------------------------------------

@router.get("/health")
async def health_check():
    """Simple readiness probe consumed by the frontend on page load."""
    return {"status": "ok", "service": "PocketSmart AI Backend"}
```

3. Design Notes

- Validation is declarative: gt=0 constraints on Field() reject non-positive income or expense values before any business logic runs.
- The calculate_totals() function is a pure function with no I/O, making it trivially unit-testable in isolation from the web framework.
- The Gemini call is isolated inside services/gemini_service.py and imported lazily, keeping the router file focused purely on HTTP concerns.
- All failures — validation errors, arithmetic edge cases, or Gemini API exceptions — are converted into structured JSON errors (422 or 502) rather than being allowed to surface as unhandled server errors.